

Eventory V2 — Customer-Side API Documentation

Covers all customer-facing backend work completed: **Phase 0 (Auth & Profile)** through **Phase 3 Step 14 (Cart & Pricing Engine)**. Generated for frontend integration.

Base URL (dev): `http://localhost:4000/api` | All request/response bodies are JSON.

0. Conventions — Read This First

Response envelope — TWO DIFFERENT SHAPES EXIST. Please handle both.

This is a real inconsistency in the codebase today, not a typo in this doc. Phase 0 (auth/profile/phone) endpoints use one envelope shape; everything built from Phase 1 onward (discovery, wishlist, compare, cart) uses a different one. Do not assume one generic response parser will work for every endpoint — check which family an endpoint belongs to.

Family	Success shape	Failure shape
Phase 0 — Auth, Profile, Phone (<code>/api/customer/auth/*</code> , <code>/api/customer/phone/*</code> , <code>/api/customers/*</code>)	<code>{ "success": true, ... }</code>	<code>{ "success": false, "message": "..."</code> <code>}</code>
Phase 1-3 — Discovery, Wishlist, Compare, Cart (everything else)	<code>{ "status": "SUCCESS", ... }</code>	<code>{ "status": "FAILED", "message": "..."</code> } (client error, e.g. 400/404/409) <code>{ "status": "ERROR", "message": "...", "error": "..."</code> } (server error, 500)

Authentication

JWT access token, issued by `POST /api/customer/auth/signup` or `/login`. Two ways to send it — use whichever fits your client:

- **Browser (recommended):** the access token is set automatically as an `httpOnly` cookie named `accessToken` on `signup/login/refresh`. Just include credentials on every request (`fetch(url, { credentials: "include" })` or your HTTP client's cookie-jar equivalent). No manual header needed.
- **Non-browser / manual testing:** send `Authorization: Bearer <accessToken>`. The same token is also returned in the JSON body of `signup/login` for this purpose.

Auth badges used throughout this doc:

- **AUTH REQUIRED** — 401s without a valid customer token.
- **AUTH OPTIONAL** — works for a logged-in customer OR an anonymous guest (cart endpoints only — see §3 Cart).
- **PUBLIC** — no token needed at all.

Rate limiting

Signup: 10/hour per IP. Login: 10/15min per IP. Phone OTP send: 5/10min. Phone OTP verify: 10/10min. All Phase 1 discovery endpoints (packages/vendors/filters/reviews): 120/min per IP.

Pagination (all list endpoints)

Query params `page` (default 1) and `limit` (default 20, max 50). Response includes `count`, `total`, `page`, `limit`, `totalPages` alongside the array.

Dates

Send dates as ISO date strings ("2026-12-25" or full ISO datetime). All day-boundary math on the server is done in UTC.

Table of Contents

- 1. Phase 0 — Auth & Profile
 - 1.1 Email/Password Auth
 - 1.2 Social Auth (Google/Facebook)
 - 1.3 Phone Verification
 - 1.4 Profile
- 2. Phase 1 — Discovery
 - 2.1 Browse Packages
 - 2.2 Package Detail (PDP)
 - 2.3 Browse Vendors
 - 2.4 Reviews
- 3. Phase 2 — Wishlist & Compare
 - 3.1 Wishlist
 - 3.2 Compare
- 4. Phase 3 — Cart & Pricing
 - 4.1 Cart CRUD
 - 4.2 Guest Cart & Merge
 - 4.3 Pricing Quote

1. Phase 0 — Auth & Profile

1.1 Email / Password Auth

POST /api/customer/auth/signup **PUBLIC**

Creates an account with name/email/password. No phone at signup — collected later via \$1.3.

Body field	Type	Notes
name	string	2–50 chars, required
email	string	valid email, required, unique
password	string	min 8 chars, required

```
201 Created
{
  "success": true,
  "message": "Account created successfully",
  "accessToken": "eyJ...",
  "customer": { "id": "CUS...", "name": "...", "email": "...", "authProviders": [...], "role": "customer",
  ... }
}
```

Errors: 409 email already registered · 400 validation (weak password, invalid email)

POST /api/customer/auth/login **PUBLIC**

Body field	Type
email	string
password	string

200 → same shape as signup. Sets accessToken + refreshToken httpOnly cookies.

Errors: 401 invalid credentials (generic message, doesn't reveal which field was wrong) · 403 account deactivated

POST /api/customer/auth/refresh-token **PUBLIC**

Reads the refresh token from the refreshToken httpOnly cookie (or { "refreshToken": "..." } in the body as a non-browser fallback). Rotates the token on every use — the old one becomes invalid immediately.

Reusing an already-rotated refresh token is treated as theft: it revokes **every** active session for that customer, not just the one being used. If you see unexpected 401s after a refresh, check you're not calling /refresh-token twice concurrently with the same stale token (e.g. two tabs racing).

200 → new accessToken (body + cookie).

POST `/api/customer/auth/logout` **PUBLIC**

Revokes the current refresh token and clears both cookies. 200 either way.

1.2 Social Auth (Google / Facebook)

Both providers 501 (not a hard failure, but not usable) until GOOGLE_CLIENT_ID/SECRET/CALLBACK_URL and/or FACEBOOK_APP_ID/SECRET/CALLBACK_URL are set in the backend's .env. Check with backend before building against these — untested end-to-end as of this doc.

GET `/api/customer/auth/google` **PUBLIC**

Redirects to Google's consent screen (standard OAuth redirect flow — navigate the browser here, don't fetch()).

GET `/api/customer/auth/google/callback` **PUBLIC**

Google redirects here after consent. On success, sets auth cookies and redirects to FRONTEND_URL (defaults http://localhost:3000). On failure, redirects to /google/failed.

Facebook: identical shape, swap google → facebook in every path above.

POST `/api/customer/auth/link-account` **AUTH REQUIRED**

Attach an additional provider to the logged-in customer's account.

Body	
provider	"google" "facebook"
providerId	string

Errors: 409 already linked to this or another account

1.3 Phone Verification

AWS Cognito OTP flow — **authenticated**, used for verifying a phone post-signup (first booking / profile), not for signup/login itself.

POST `/api/customer/phone/send-otp` **AUTH REQUIRED**

Body	
mobile	10-digit Indian mobile, e.g. "9876543210"

Errors: 409 phone already linked to another account

POST `/api/customer/phone/verify-otp` **AUTH REQUIRED**

Body	
mobile	same 10-digit number
code	4-8 digit OTP
session	string, returned by send-otp's underlying Cognito call

1.4 Profile

GET `/api/customers/:id` AUTH REQUIRED (self only)

`:id` must equal the logged-in customer's own id (e.g. CUS20260807...) — 403 otherwise. Note the base path is `/api/customers`, not `/api/customer/...`

PATCH `/api/customers/:id` AUTH REQUIRED (self only)

Editable: name, profilePicture (URL, not a data-URI), dateOfBirth, gender, addresses[], preferences. Identity/role/ verification fields cannot be changed here.

PATCH `/api/customers/:id/deactivate` AUTH REQUIRED (self only)

2. Phase 1 — Discovery

All endpoints in this section are PUBLIC (no auth). Only packageStatus: "Live" packages / non-deactivated vendors are ever returned.

2.1 Browse Packages

GET /api/customer/packages

Query param	Type	Notes
eventCategory	string	e.g. "Wedding"
vendorType	enum	Caterer Decorator PAV DJArtist MakeupArtist VenueProvider
city	string	matched against the owning vendor's city/serviceAreas
guests	int	must fall within the package's capacity range
date	date	excludes packages Blocked/Booked that day
minPrice / maxPrice	number	
sort	enum	newest (default) price_asc price_desc rating
page / limit	int	see \$0 Pagination

```
200 { "status": "SUCCESS", "count", "total", "page", "limit", "totalPages",
      "packages": [ { "_id", "vendorId": { public vendor fields }, "vendorType", "variantType",
        "step1_eventAndCrew": {...}, "step3_policiesAndCharges": { "packagePricing", "gstInclusive",
        "gstRatePercent", "guestTiers" },
        "step4_sampleMedia": { "media": [...] }, "createdAt" } ] }
```

GET /api/customer/packages/filters

Distinct eventCategories/vendorTypes/cities + price/guest range actually present in Live inventory right now — NOT a hardcoded master list (none exists in this backend). Also returns sortOptions.

2.2 Package Detail (PDP)

GET

/api/customer/packages/:packageId

Query (all optional)	
date	date
guests	int
time	"HH:MM" 24h

404 if not found or not Live. Returns the full package plus 3 computed sections:

```
{
  "package": { ...full package doc, vendor populated... },
  "availability": {
    "guests", "withinCapacity", "date", "calendarStatus" ("Available"|"Blocked"|"Booked"|"Unknown"),
    "matchesWeeklyAvailability", "activeBookingsOnDate", "dailyCapacity", "capacityAvailable",
    "time", "matchesDeclaredTimeSlot",
    "overall": true | false | null  // null = not enough signals to say either way, NOT a "yes"
  },
  "pricingPreview": {
    "basePrice", "billingUnit", "guestTierApplied", "appliedAdjustment", "unappliedToggles": [...],
    "subtotal", "gstInclusive", "gstRatePercent", "gstAmount", "total",
    "note": "Preview estimate only – never trusted server-side at checkout"
  },
  "reviews": { "items": [...], "count", "averageRating", "usingVendorFallback": bool }
}
```

pricingPreview is an ESTIMATE for display only — do not use it to show a "final" price anywhere near a payment action. Always re-fetch the authoritative number from the Cart quote endpoint (\$4.3) once the item is in cart.

2.3 Browse Vendors

GET

/api/customer/vendors

Query	
vendorType, eventCategory, city	string
sort	rating (default) newest
page / limit	int

Response fields are an explicit whitelist — **never** includes phone, email, aadhar/pan/gst numbers, bank details, or internal KYC flags. Safe to render directly.

```
"vendors": [ { "id", "businessName", "isIndividual", "vendorType", "eventCategories", "city", "state",
  "serviceAreas", "teamSize", "bookingsPerYear", "experience", "profilePicture", "description",
  "businessPhotos", "coverImage", "isVerified", "rating", "reviewsCount", "createdAt" } ]
```

GET

/api/customer/vendors/filters

Same "derived from real data" pattern as packages/filters.

2.4 Reviews

GET /api/customer/packages/:packageId/reviews

Query	
minRating	1-5
sort	recent (default) highest lowest
page / limit	int

```
{ "items": [...], "total", "aggregate": {
  "averageRating", "count", "distribution": { "1":n,"2":n,"3":n,"4":n,"5":n},
  "categoryBreakdown": [ { "category", "averageRating", "count" } ] } }
```

GET /api/customer/vendors/:vendorId/reviews

Same shape as above, scoped to every review a vendor has received across all their packages.

There is currently very little real review data in the dev database — expect mostly empty `items` arrays / fallback aggregates during integration testing. This is expected, not a bug.

3. Phase 2 — Wishlist & Compare

Everything in this section is **AUTH REQUIRED** (except the one explicitly public shared-wishlist read below).

3.1 Wishlist

GET `/api/customer/wishlist`

Package items include a computed priceChanged flag (current price vs. price when saved) and packageStillAvailable. This is passive/computed-on-read — there is no push/email price-drop alert system.

POST `/api/customer/wishlist`

Body	
itemType	"Package" "Vendor"
packageId	required if itemType=Package
vendorId	required if itemType=Vendor
note	string, max 500

Errors: 404 not found/unavailable · 409 already saved

PATCH `/api/customer/wishlist/:itemId`

Body: { "note": "..." }

DELETE `/api/customer/wishlist/:itemId`

DELETE `/api/customer/wishlist`

Clear all.

POST `/api/customer/wishlist/share?rotate=`

Mints (or, with rotate=true, replaces) a share token. Returns { shareToken, shareUrl }.

DELETE `/api/customer/wishlist/share`

Disables sharing; token is kept (re-enabling reuses the same link).

GET `/api/customer/wishlist/shared/:token` **PUBLIC**

Read-only public view. **Personal notes are deliberately excluded** from this response.

3.2 Compare

Packages only (not vendors). Max 3 items, and all items in one session must share the same vendorType — adding a package from a different category 409s.

GET /api/customer/compare

DELETE /api/customer/compare

POST /api/customer/compare/items

Body: { "packageId": "..."} }

Errors: 404 not found · 409 duplicate / full (max 3) / vendorType mismatch

DELETE /api/customer/compare/items/:packageId

GET /api/customer/compare/export?format=

format=json (default, same shape as GET) or format=csv (triggers a file download via Content-Disposition). 400 if the comparison is empty.

4. Phase 3 — Cart & Pricing Engine

4.1 Cart CRUD

Every cart endpoint accepts **EITHER a logged-in customer OR an anonymous guest** — see §4.2 for how the guest flow works. Read this before building the cart UI.

GET `/api/customer/cart`

Never creates a cart from a mere read. No identity + no existing cart → empty payload, zero DB writes.

```
{ "cartId", "itemCount", "vendorCount",
  "vendors": [ { "vendorId", "vendorSubtotal", "items": [ {
    "_id", "packageId", "packageSnapshot": {name, price, billingUnit, image, vendorType, variantType},
    "eventDetails": {date, timeSlot, guestCount, location},
    "selectedAddOns": [...], "selectedItems": [...], "specialRequest", "quantity",
    "selectedForCheckout": bool,
    "packageStillAvailable", "currentPrice", "priceChanged",
    "availability": { ...same shape as PDP's availability... },
    "lineTotal"
  } ] } ],
  "coupon", "bookingNote", "subtotal", "discount", "total",
  "conflicts": { "inconsistentEventDates": bool, "dates": [...] }
}
```

subtotal/total/each vendorSubtotal only sum items where selectedForCheckout: true. Unselected items still appear in items[] (render them, just excluded from the money shown) — this matches the "select a subset to check out" cart UX.

DELETE `/api/customer/cart`

Clears all items. Coupon/booking note are untouched (clear those separately).

POST `/api/customer/cart/items`

Body	
packageId	required
guests	int — 400s if outside the package's own capacity range (the only hard block)
date	date
timeSlot, location	string
selectedAddOns	[{ addOnId?, name, price, quantity }]
selectedItems	[{ groupKey, itemId?, itemName, price, isChargeable }] — choose-N menu selections
specialRequest	string, max 500 — per-vendor note
quantity	int, default 1

201 → full cart payload (same shape as GET), plus `itemId`. See §4.2 for the guest-cart-id fields.

A Blocked/Booked date does **not** block adding to cart — it's surfaced via the item's availability on GET instead, same as the PDP. Only capacity mismatch and non-Live packages are hard 404/400s.

PATCH `/api/customer/cart/items/:itemId`

Partial update — send only the fields you want to change: `guests`, `date`, `timeSlot`, `location`, `selectedAddOns`, `selectedItems`, `specialRequest`, `quantity`, `selectedForCheckout` (the select-for-checkout checkbox).

DELETE `/api/customer/cart/items/:itemId`**POST** `/api/customer/cart/items/:itemId/move-to-wishlist` **AUTH REQUIRED**

The one cart endpoint that's customer-only, not guest-usable (guests have no wishlist). Removes from cart either way, even if it was already wishlisted earlier.

POST `/api/customer/cart/coupon`

Body: { "code": "..."}

Placeholder, not real yet: there is no Coupon/Offer system in this backend. This stores the code and always applies a **0 discount** — the response message says so explicitly. Wire the UI to this endpoint now; the discount math will start working once a real coupon system exists, with no API shape change needed.

DELETE `/api/customer/cart/coupon`**PUT** `/api/customer/cart/note`

Body: { "note": "..."} — one global note for the whole order, distinct from each item's own `specialRequest`.

4.2 Guest Cart & Merge

A cart can exist before login. There is no session/cookie for this — the client owns a `guestCartId` string.

1. First POST `/api/customer/cart/items` with no logged-in session and no guest id sent → server mints one. It comes back two ways: JSON body field `"guestCartId": "..."`, and response header `X-Guest-Cart-Id`. **Store this** (e.g. `localStorage`).
2. Every subsequent cart request (any method) while anonymous: send header `X-Guest-Cart-Id: <value>` (or ? `guestCartId=` query param, mainly for quick manual testing).
3. When the user logs in, call:

POST `/api/customer/cart/merge` **AUTH REQUIRED**

Body: { `"guestId": "<the stored guestCartId>"` }

Moves every item from the guest cart into the now-logged-in customer's own cart, adopts the guest cart's coupon/note only if the customer cart didn't already have its own, then deletes the guest cart. Returns the merged cart (same shape as GET). Safe to call with an already-merged/unknown `guestId` — responds 200 "nothing to merge" rather than erroring.

This is deliberately a manual step, not automatic on login — call it explicitly right after login completes. Do not call it speculatively on every page load.

4.3 Pricing Quote

GET

/api/customer/cart/quote

The **authoritative** price/due-now breakdown — use this right before showing a payment amount anywhere, not the plain GET /cart totals (which are for display and price off a cached snapshot) and not the PDP's pricingPreview (which is a pre-cart estimate). Only items with selectedForCheckout: true are priced. Returns { "quote": null } if there's no cart or nothing selected.

```
"quote": {
  "lines": [ {
    "cartItemId", "vendorId", "packageId", "available": true,
    "snapshotPrice", "currentPrice", "priceChanged",
    "packagePriceTotal", "addonsTotal", "chargeableItemsTotal", "lineSubtotal",
    "gstInclusive", "gstRatePercent", "gstAmount", "lineTotalInclGst",
    "token": { "tokenConfigured", "paymentType" ("Free"|"Token"), "tokenType"?, "tokenValue"?,
"tokenAmount", "remaining" },
    "milestones": [ { "title", "percentage", "amount", "dueDaysRaw", "dueDate" } ]
  } ],
  "subtotal", "gstTotal",
  "convenienceFeePercent", "convenienceFeeConfigured", "convenienceFee",
  "discount", "grandTotal",
  "tokenAmountTotal", "remainingTotal", "allTokensConfigured", "note"
}
```

Field	What it means for the UI
priceChanged (per line)	Price moved since it was added to cart — consider prompting the user to confirm before payment.
available: false (per line)	Package is no longer Live. A note field explains it. Prompt the user to remove/replace this item — it's excluded from all totals.
token.tokenConfigured: false	That vendor hasn't set up an advance-payment config. Never assume 25% or any other percentage in the UI for this case — show it as "pending" / unavailable.
allTokensConfigured: false	tokenAmountTotal/remainingTotal are both null cart-wide when even one selected line lacks token config — note explains why.
convenienceFeeConfigured: false	No fee % has been set on the backend (CONVENIENCE_FEE_PERCENT env var) — fee shows as 0, not "unknown". Confirm with backend/finance before launch whether this should be a real, non-zero number.
milestones[].dueDate: null	The vendor entered a non-numeric due-date label (e.g. "On event day") — show dueDaysRaw as text instead of a computed date.

Appendix — Known Gaps / Things Not Yet Built

So the frontend team doesn't spend time debugging something that's simply not implemented yet:

- **No Coupon/Offer system** — see §4.1, coupon apply is a stored-but-inert placeholder.
- **No checkout/payment endpoints yet** — Phase 4 (Cashfree integration, booking creation) has not started. Cart's /quote is as far as the backend goes today.
- **No Essentials model** — only Packages and Vendors exist as browsable/cart-able item types.
- **No PDP-friendly slug routes** — package/vendor detail routes use MongoDB ObjectIds (:packageId, :vendorId), not SEO slugs.
- **Google/Facebook OAuth is untested end-to-end** — code is complete but needs real provider credentials in the backend's .env to actually exercise.
- **Email verification and forgot-password are not built** — no email-sending capability exists yet.
- **Response envelope is inconsistent** between Phase 0 and everything after — see §0, top of this document.